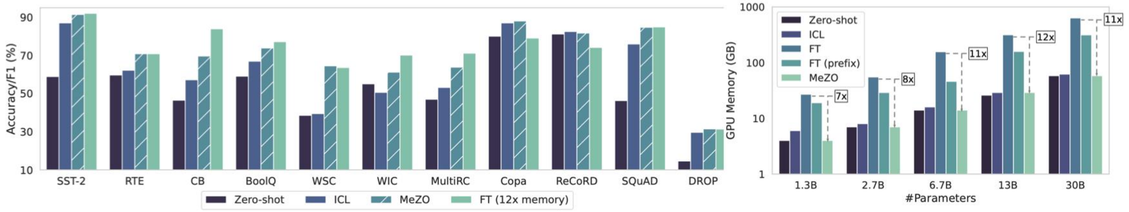
Fine-Tuning Language Models with Just Forward Passes

Sadhika Malladi*, Tianyu Gao*, Eshaan Nichani, Alex Damian, Jason D. Lee, Danqi Chen, Sanjeev Arora. **Princeton University**

[1] J.C. Spall. *Multivariate stochastic approximation using a simultaneous perturbation gradient approximation.* IEEE Transactions on Automatic Control (1992).
[2] Hu et al. *LoRA: Low-rank adaptation of large language models.* (ICLR '22)
[3] Li & Liang. *Prefix Tuning: Optimizing Continuous Prompts for Generation.* (ACL '21)

Fine-tuning LLMs with **backpropagation** requires a prohibitively **large amount of memory**.
We propose **MeZO**, a memory-efficient zeroth-order optimizer.
Costs the **same memory as inference without sacrificing much performance**.
With one A100, MeZO can tune a **30B** model while Adam can only tune a **2.7B** model.



Left: OPT-13B performance on classification, multiple-choice, and generation tasks.
Right: memory consumption of different-sized OPT models. ICL: in-context learning; FT: fine-tuning with Adam.

MeZO Algorithm

Classic zeroth-order (**ZO**) gradient estimate with two forward passes: **SPSA** [1]

$$\hat{\nabla} L(\theta; B) = \frac{L(\theta + \epsilon z; B) - L(\theta - \epsilon z; B)}{2\epsilon} z \approx z z^\top \nabla L(\theta; B)$$

Memory-efficient ZO-SGD (MeZO): resample z so SGD with SPSA uses no extra memory besides inference cost

(by first sampling a **random seed** and later restoring the random seed to make sure the sampled z is consistent)

Algorithm 1: MeZO

Require: parameters $\boldsymbol{\theta} \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$, step budget T, perturbation scale ϵ, batch size B learning rate schedule $\{\eta_t\}$

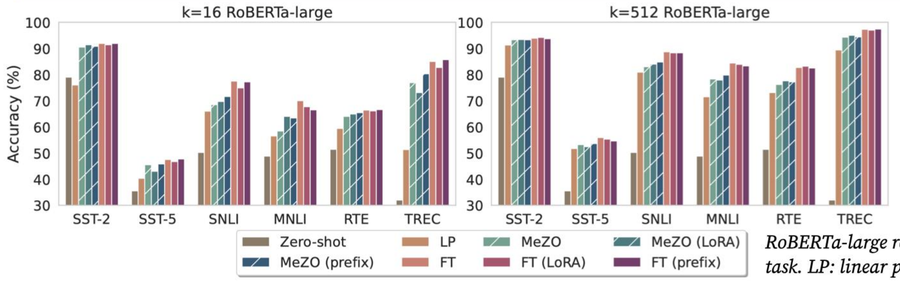
for $t = 1, \ldots, T$ **do**
  Sample batch $\mathcal{B} \subset \mathcal{D}$ and random seed s
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters`$(\boldsymbol{\theta}, \epsilon, s)$
  $\ell_+ \leftarrow \mathcal{L}(\boldsymbol{\theta}; \mathcal{B})$
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters`$(\boldsymbol{\theta}, -2\epsilon, s)$
  $\ell_- \leftarrow \mathcal{L}(\boldsymbol{\theta}; \mathcal{B})$
  $\boldsymbol{\theta} \leftarrow$ `PerturbParameters`$(\boldsymbol{\theta}, \epsilon, s)$ ▷ `Reset parameters before descent`
  `projected_grad` $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$
  Reset random number generator with seed s ▷ `For sampling` z
  for $\theta_i \in \boldsymbol{\theta}$ **do**
    $z \sim \mathcal{N}(0, 1)$
    $\theta_i \leftarrow \theta_i - \eta_t *$ `projected_grad` $* z$
  end
end

Subroutine `PerturbParameters`$(\boldsymbol{\theta}, \epsilon, s)$
  Reset random number generator with seed s ▷ `For sampling` z
  for $\theta_i \in \boldsymbol{\theta}$ **do**
    $z \sim \mathcal{N}(0, 1)$
    $\theta_i \leftarrow \theta_i + \epsilon z$ ▷ `Modify parameters in place`
  end
  return $\boldsymbol{\theta}$

Why does it work?
Classical results: ZO slows down by **number of parameters** d
Our case: **fine-tuning** a pre-trained model with a **prompt**, so that fine-tuning looks like pre-training!
Then: **effective rank of Hessian** r is small, convergence rates depend on r instead of d (descent lemma)

Experiments

Our examples cover **medium-sized masked LMs** (RoBERTa-large) and **large autoregressive LMs** (OPT-13B, 30B, 66B).
Both **full-parameter** and **parameter-efficient** fine-tuning (e.g., LoRA, prefix-tuning).
We experiment on **classification, multiple-choice, and generation** tasks.



RoBERTa-large results. We sample 512 examples for each class in each task. LP: linear probing; FT: fine-tuning.

1. On RoBERTa-large, MeZO outperforms **zero-shot and linear probing.**
2. MeZO is compatible with both full-parameter and **parameter-efficient fine-tuning** (PEFT), such as LoRA [2], prefix-tuning [3].

Task	SST-2	RTE	CB	BoolQ	WSC	WIC	MultiRC	COPA	ReCoRD	SQuAD	DROP
Task type	classification							– multiple choice –		— generation —	
Zero-shot	58.8	59.6	46.4	59.0	38.5	55.0	46.9	80.0	81.2	46.2	14.6
ICL	87.0	62.1	57.1	66.9	39.4	50.5	53.1	87.0	**82.5**	75.9	29.6
LP	**93.4**	68.6	67.9	59.3	63.5	60.2	63.5	55.0	27.1	3.7	11.1
MeZO	91.4	66.1	67.9	67.6	63.5	**61.1**	60.1	**88.0**	81.7	**84.7**	30.9
MeZO (LoRA)	89.6	67.9	66.1	**73.8**	**64.4**	59.7	61.5	87.0	81.4	83.8	**31.4**
MeZO (prefix)	90.7	**70.8**	**69.6**	73.1	57.7	59.9	**63.7**	84.0	81.2	84.2	28.9
FT (12x memory)	92.0	70.8	83.9	77.1	63.5	70.1	71.1	79.0	74.1	84.9	31.3

OPT-13B results. Each task has 1,000 examples. ICL: in-context learning.

3. On OPT-13B, MeZO achieves **on par performance with Adam** on 7/11 task, while **saving up to 12x memory**.
4. MeZO also beats memory-equivalent methods like **in-context learning** and linear probing.

Task	SST-2	RTE	BoolQ	WSC	WIC	SQuAD
30B zero-shot	56.7	52.0	39.1	38.5	50.2	46.5
30B ICL	81.9	66.8	66.2	56.7	51.3	78.0
30B MeZO/MeZO (prefix)	**90.6**	**72.6**	**73.5**	**63.5**	**59.1**	**85.2**
66B zero-shot	57.5	**67.2**	66.8	43.3	50.6	48.1
66B ICL	89.3	65.3	62.8	52.9	54.9	81.3
66B MeZO/MeZO (prefix)	**93.6**	66.4	**73.7**	**63.5**	**58.9**	**85.0**

OPT 30B/66B results. We report the best of MeZO/MeZO (prefix).

5. MeZO can **scale up to 66B** models.

Model	RoBERTa-large (350M)				OPT-13B
Task	SST-2	SST-5	SNLI	TREC	SQuAD
Zero-shot	79.0	35.5	50.2	32.0	46.2
Cross entropy (FT)	93.9	55.9	88.7	97.3	84.2
Cross entropy (MeZO)	93.3	53.2	83.0	94.3	84.7
Accuracy/F1 (MeZO)	92.7	48.9	82.7	68.6	78.5

For classification (k=512), we use MeZO with full parameters and optimize accuracy; for SQuAD (1,000 examples), we use MeZO (prefix) and F1.

6. MeZO works with **non-differentiable** objectives.

MeZO can also beat fine-tuning on wall clock time at scale. Check out out paper for more details!

When should you use MeZO

efficient, fuzzy gradient estimate (e.g., data selection)
optimize **non-differentiable objectives** (e.g., with a reward model)
study **huge models** but don't have many GPUs

When shouldn't you use MeZO

Does fine-tuning a model do X?
I have a new task without obvious prompts
I only want to study small models